What is R?

"**R** is an integrated suite of software facilities for **data manipulation**, **calculation** and **graphical display**"

Dialect of the **S** language developed by **John Chambers** and others at the **Bell Labs** in **1976**

Created in New Zealand by **Ross Ihaka** and **Robert Gentleman** in **1991.** In **1997 R Core** group is formed who control the source code for R.

Version 1.0.0 February 2000

Version 2.0.0 October 2004

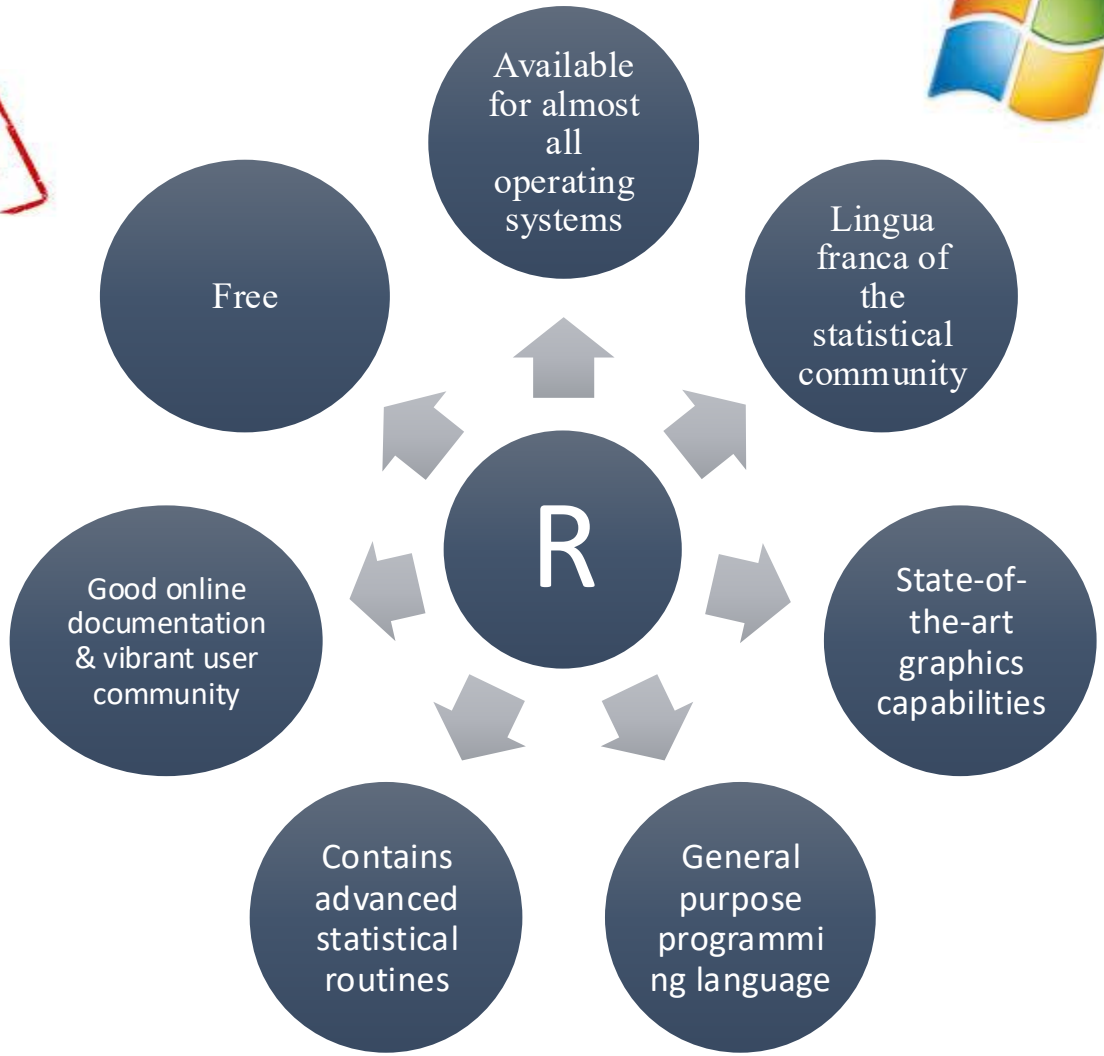
Version 4.6.0 May 2026

Why R?


100% FREE


stackoverflow


reshape
ggplot2
plyr
data.table
reshape2
randomForest

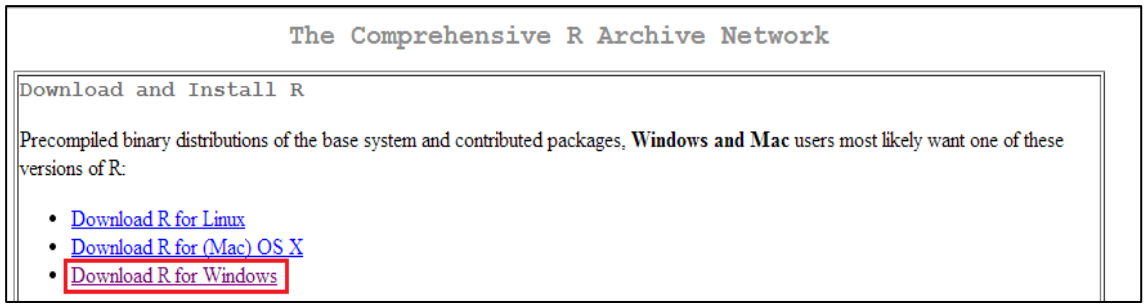

R
Free
Available for almost all operating systems
Lingua franca of the statistical community
State-of-the-art graphics capabilities
General purpose programming language
Contains advanced statistical routines
Good online documentation & vibrant user community

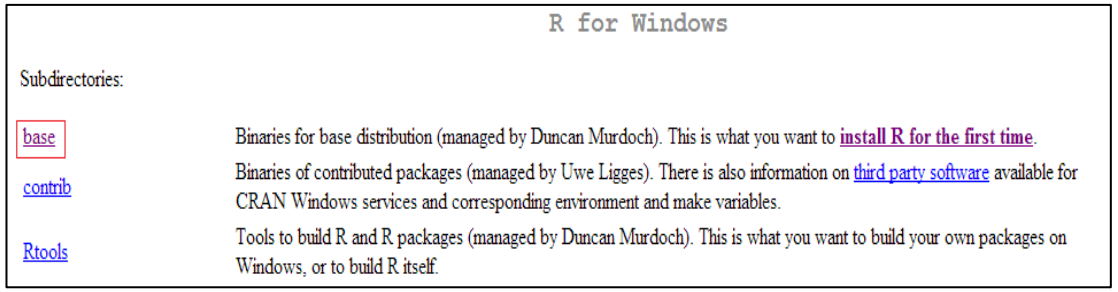

Statistics
organization
descriptive
collection
data
interpretation
statistical
Inference
theory
mathematics

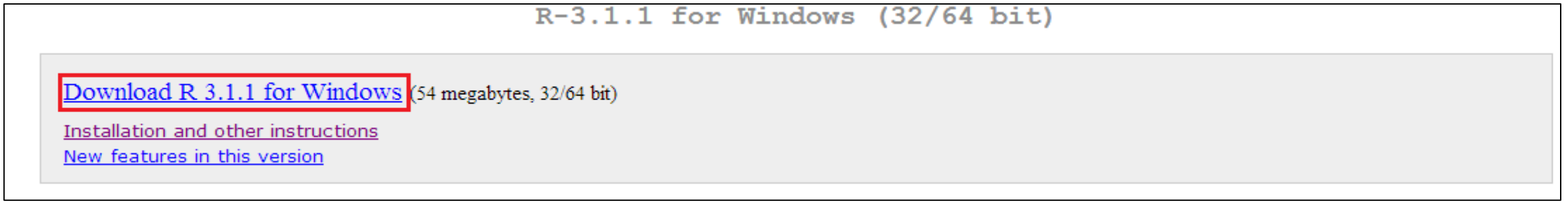

facebook

Download R

Visit https://lib.stat.cmu.edu/R/CRAN/

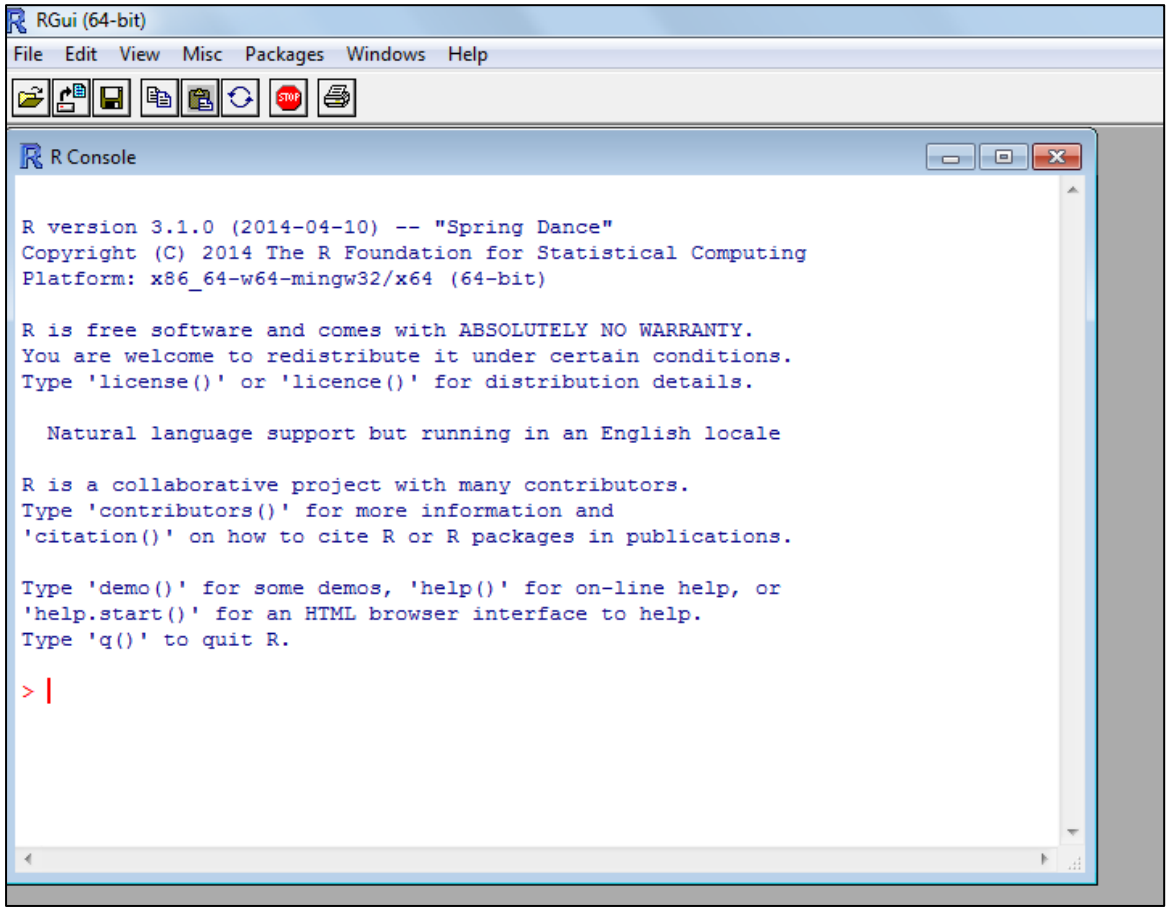


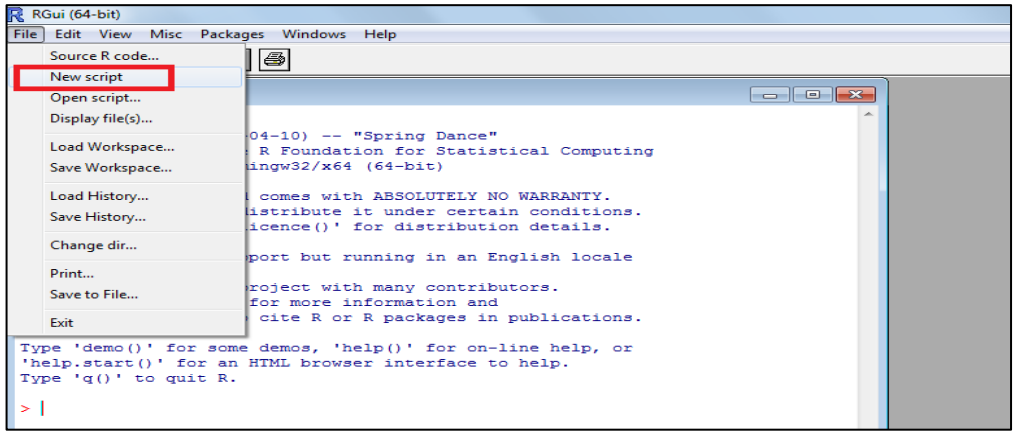


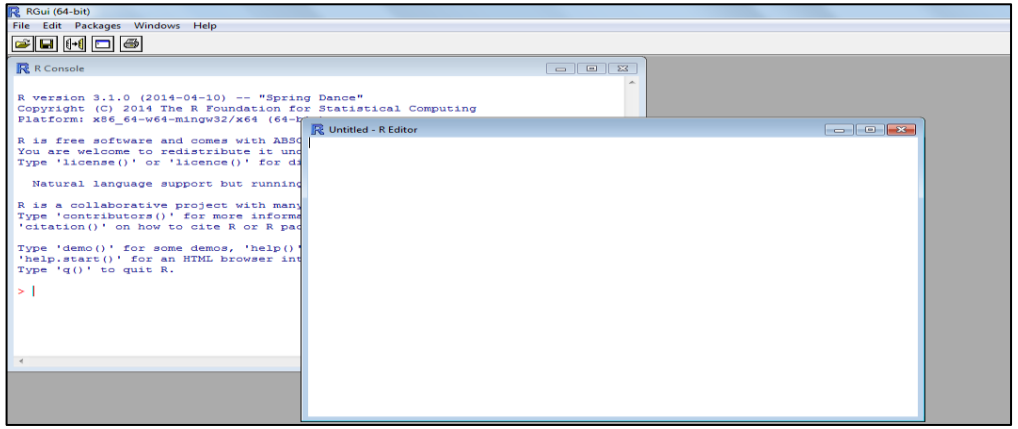


For downloading Rstudio go to http://www.rstudio.com/

R Interface


RGui (64-bit)
File Edit View Misc Packages Windows Help
R Console
R version 3.1.0 (2014-04-10) -- "Spring Dance"
Copyright (C) 2014 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
Natural language support but running in an English locale
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
>


RGui (64-bit)
File Edit View Misc Packages Windows Help
Source R code...
New script
Open script...
Display file(s)...
Load Workspace...
Save Workspace...
Load History...
Save History...
Change dir...
Print...
Save to File...
Exit
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
>


RGui (64-bit)
File Edit Packages Windows Help
R Console
Untitled - R Editor

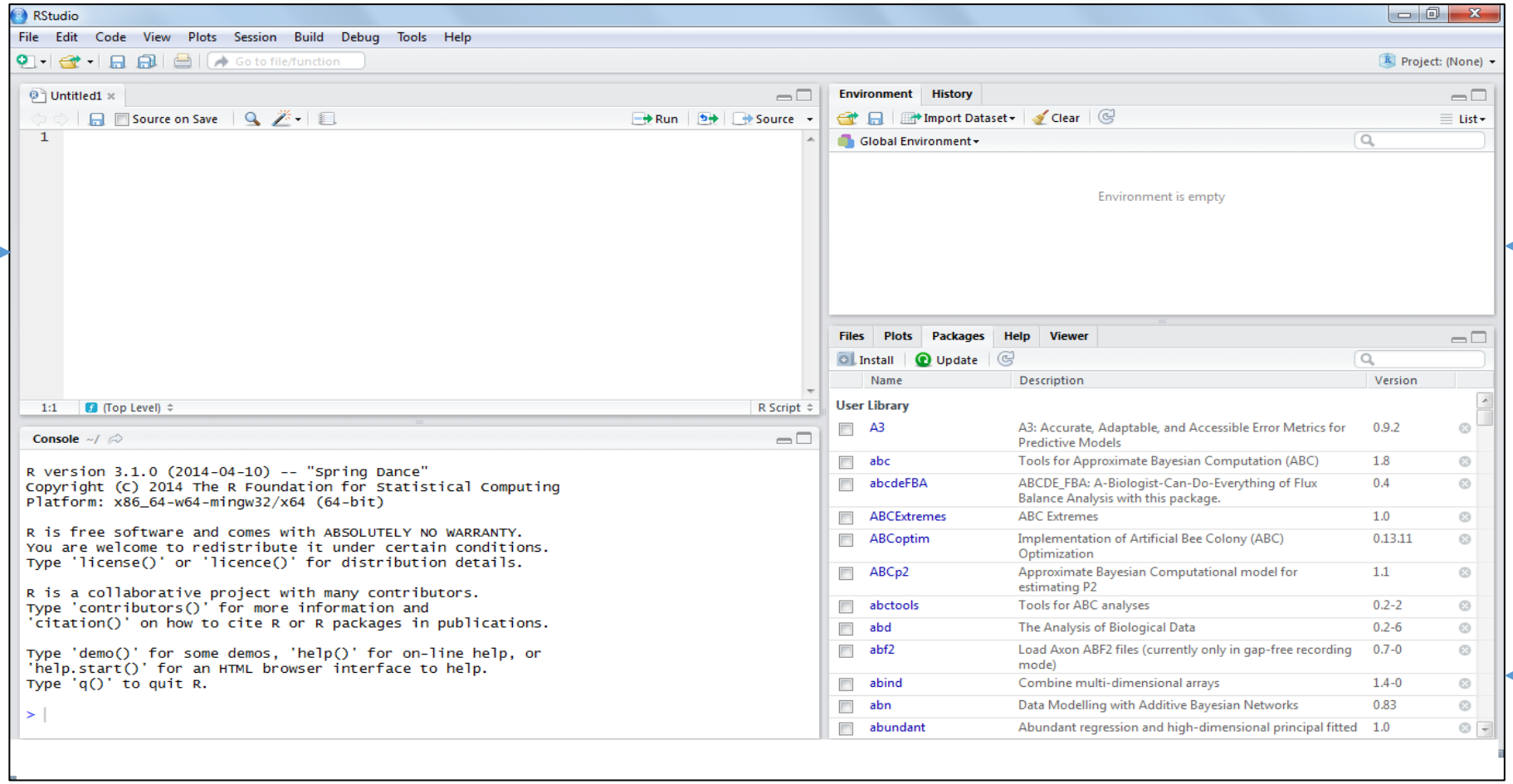
RStudio Interface

R Script

R Console

History/ Workspace

Help/Packages/Plots



Graphic User Interfaces

R provides simple **Command line interface(CLI)** where user enters statements at command prompt (> by default) and each command is executed one at a time. This is a **limitation** of R.

So, many GUIs ranging from syntax editors to full blown GUIs for doing analysis with drop down menus available

Integrated development environments and syntax editors

Name	URL
Eclipse with StatET plug-in	http://www.eclipse.org and http://www.walware.de/goto/statet
ESS (Emacs Speaks Statistics)	http://ess.r-project.org/
Komodo Edit with SciViews-K plug-in	http://www.activestate.com/komodo_edit/ http://www.sciviews.org/SciViews-K/
JGR	http://www.rforge.net/JGR/
RStudio	http://www.rstudio.org
Tinn-R (Windows only)	http://www.sciviews.org/Tinn-R/
Notepad++ with NppToR (windows only)	http://notepad-plus-plus.org/ http://sourceforge.net/projects/npptor/

Comprehensive GUIs

Name	URL
JGR/ Deducer	http://ifellows.ucsd.edu/pmwiki/pmwiki.php?n=Main.DeducerManua
R AnalyticFlow	http://www.ef-prime.com/products/ranalyticflow_en/
Rattle (for data mining)	http://rattle.togaware.com/
R Commander	http://socserv.mcmaster.ca/jfox/Misc/Rcmdr/
Red R	http://www.red-r.org/
Rkward	http://rkward.sourceforge.net/

First Session

- R is a **interpreted**, **command based** language
- R is **case sensitive**
- Statements need not end with a semicolon. However if multiple statements are written in a line, semicolon is required.
- Comments begin with # symbol
- Results of calculations can be stored in objects using the assignment operators:
  - An arrow (<-) formed by a smaller than character and a hyphen without a space
  - The equal character (=)
- Object names cannot contain `strange' symbols like !, +, -, #
  - **A dot (.)** and **an underscore (_)** are allowed, also a name starting with a dot.
  - Object names can contain a number but cannot start with a number
- Explicit print statements are not necessary
- Script files are **saved** with **.R** extension
- Quit R using **q()** command or **File menu->Exit**

Getting Help

Plethora of resources are available to help us learn about R. This include several facilities within R and the Web

R Internal Help

Function	Action
help.start()	Getting help
help("foo") or ?foo	Help on function foo (the quotation marks are optional)
help.search("foo") or ??foo	Search the help system for instances of the string foo
help(package=foo)	Learn about entire package
example("foo")	Examples of function foo (the quotation marks are optional)
RSiteSearch("foo")	Search for the string foo in online help manuals and archived mailing lists
apropos("foo", mode="function")	List all available functions with foo in their name
data()	List all available example datasets contained in currently loaded packages
vignette()	List all available vignettes for currently installed packages
vignette("foo")	Display specific vignettes for topic foo

R Internet Help

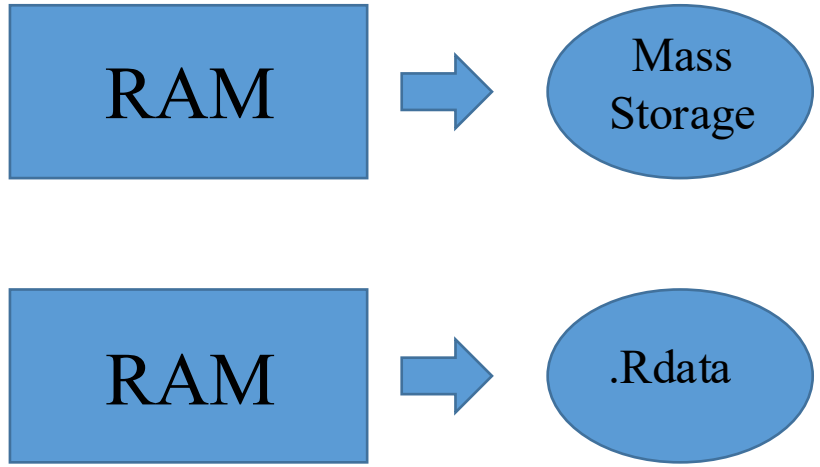
- ✓ Manuals available from http://www.r-project.org/
- ✓ Search engines listed on R home page. Click Search
- ✓ **"sos"** package in R
- ✓ Rseek search engine: http://www.rseek.org/
- ✓ http://stackoverflow.com/
- ✓ http://r.789695.n4.nabble.com/

R Workspace

Objects (vectors, matrices, functions, data frames, or lists). that you create during an R session are held in **memory,** the collection of objects that you currently have is called the workspace. This workspace is not saved on disk unless you tell R to do so.

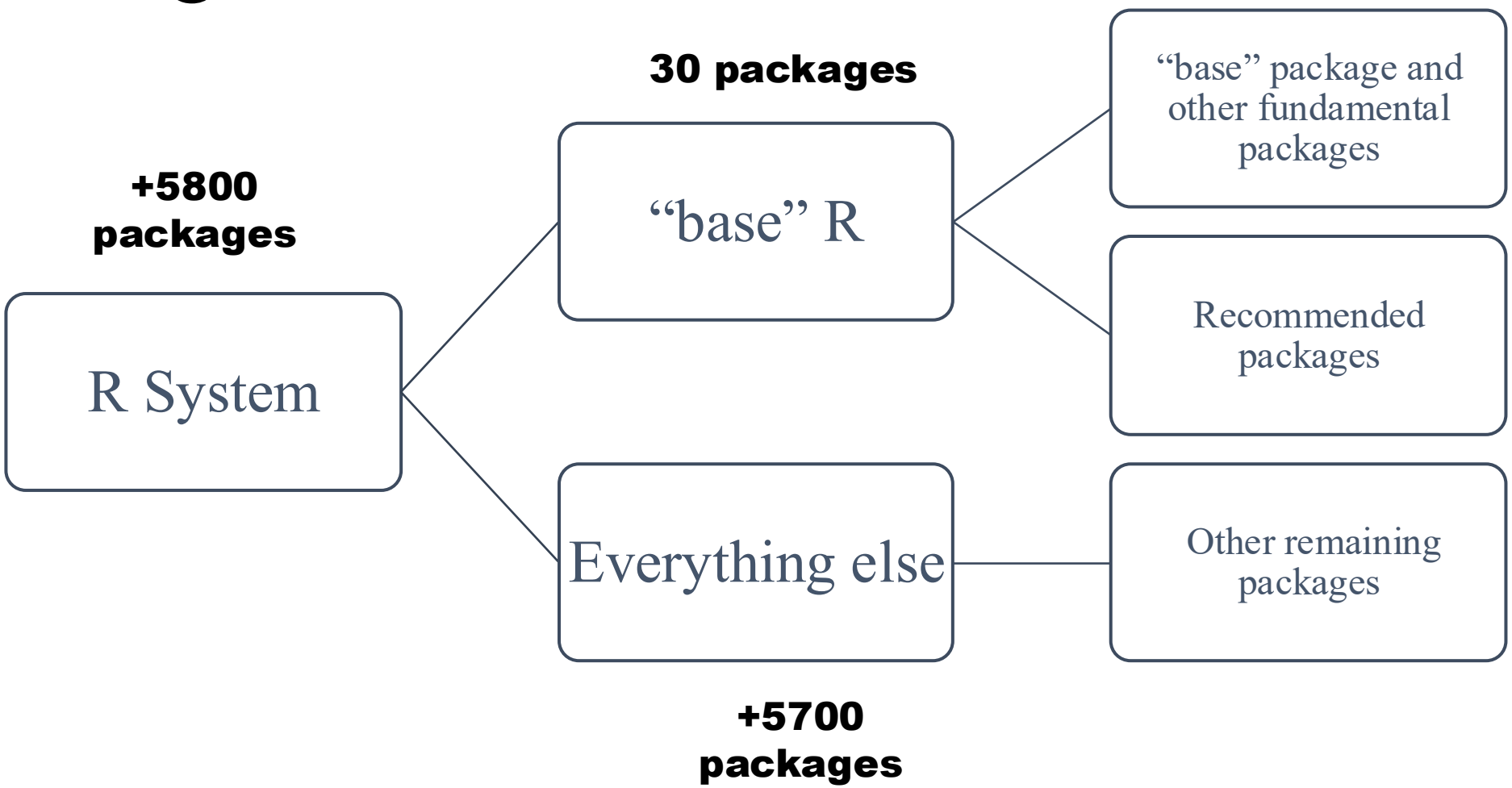
Functions for managing R Workspace

Function	Action
getwd()	List the current working directory
setwd("mydirectory")	Change the current working directory to mydirectory
ls()	List the objects in the current workspace
rm(objectlist)	Remove (delete) one or more objects
help(options)	Learn about available options
options()	View or set current options
history(#)	Display your last # commands (default = 25)
savehistory("myfile")	Save the commands history to myfile (default =.Rhistory)
loadhistory	("myfile") Reload a command's history (default = .Rhistory).
save.image("myfile")	Save the workspace to myfile (default = .RData).
save(objectlist,file="myfile")	Save specific objects to a file.
load("myfile")	Load a workspace into the current session (default =.RData)



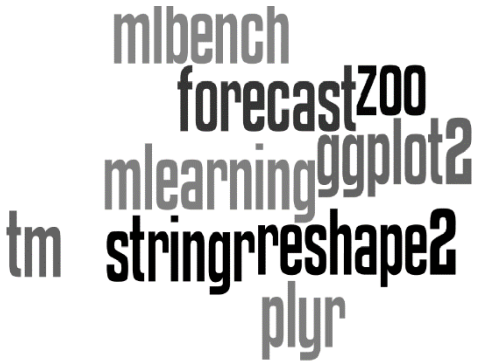
The workspace is saved as a file called .Rdata and when R starts up, it checks for such a file in the current working directory and loads it automatically

Design of R


+5800 packages
R System
30 packages
"base" R
Everything else
"base" package and other fundamental packages
Recommended packages
Other remaining packages
+5700 packages


grid
tcltk
compiler
splines
graphics
grDevices
base
parallel
datasets
stats
stats4
methods

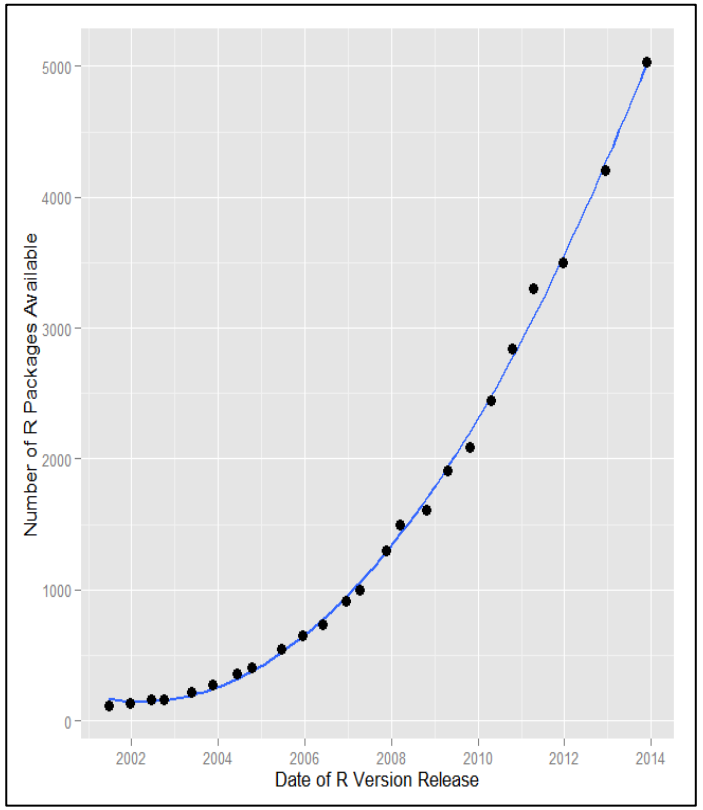

nnet
classspatial
boot
cluster
survival
Matrix
KernSmooth
codetools
rpart
MASS
lattice
foreign
mgcv
nlme


mlbench
forecast
zoo
mlearning
ggplot2
tm
stringr
reshape2
plyr

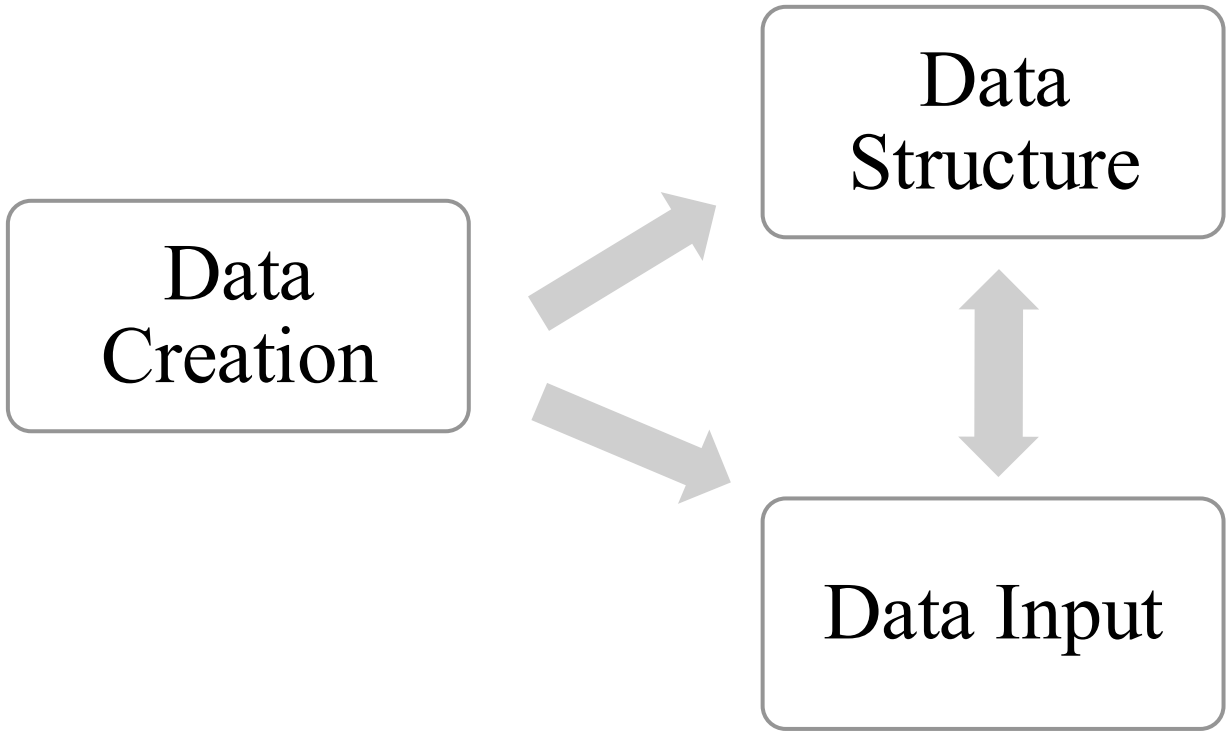
R Packages

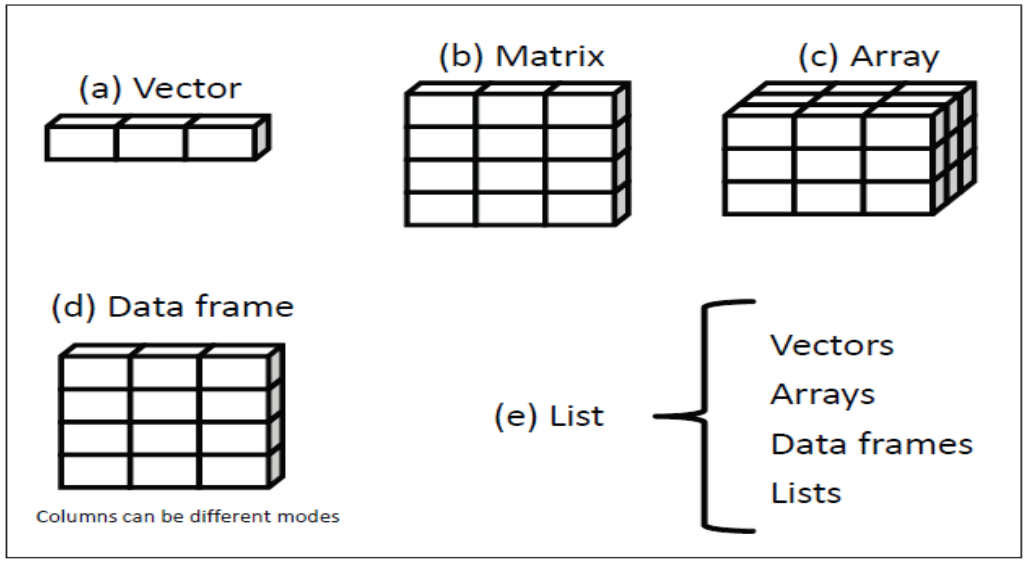
Functionality of R is divided into a number of packages. Packages are collections of R functions, data, and compiled code in a well-defined format. The directory where packages are stored on your computer is called the library.

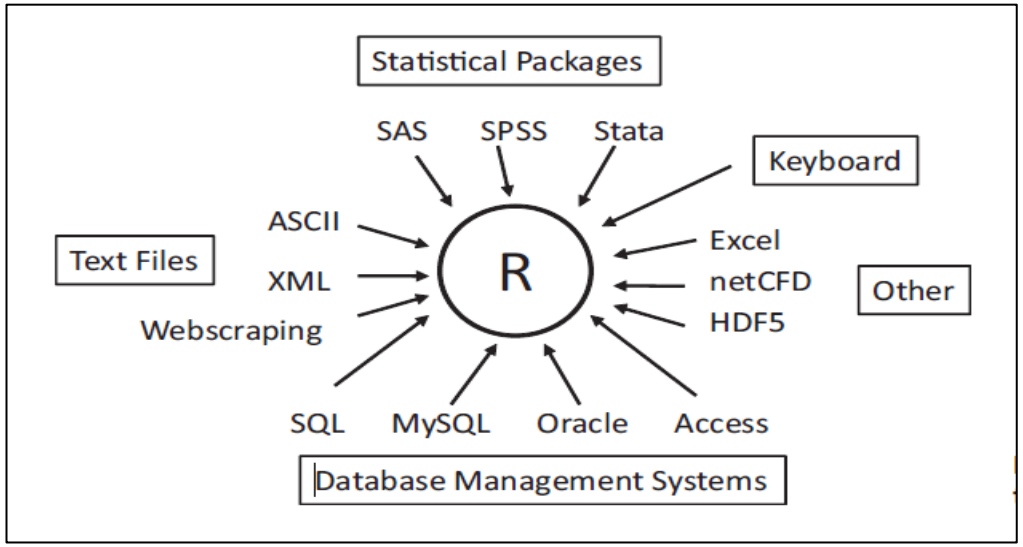
Function	Action
.libPaths()	Where library is located
library()	Shows packages stored in
search()	Which packages are loaded
install.packages("packagename",dependencies=TRUE)	Loads required package as well as dependent packages
update.packages("packagename")	Updates required package
library(packagename)/require(packagename)	Load package
help(package="package_name")	Help about required package
detach("package:package_name",unload=TRUE)	Unload package
remove.packages("packagename")	Permanently delete package
ls("package:package_name", all = TRUE)	Shows all functions and datasets in package
data(package = "packagename")$results	all datasets with brief description in package



Data Creation





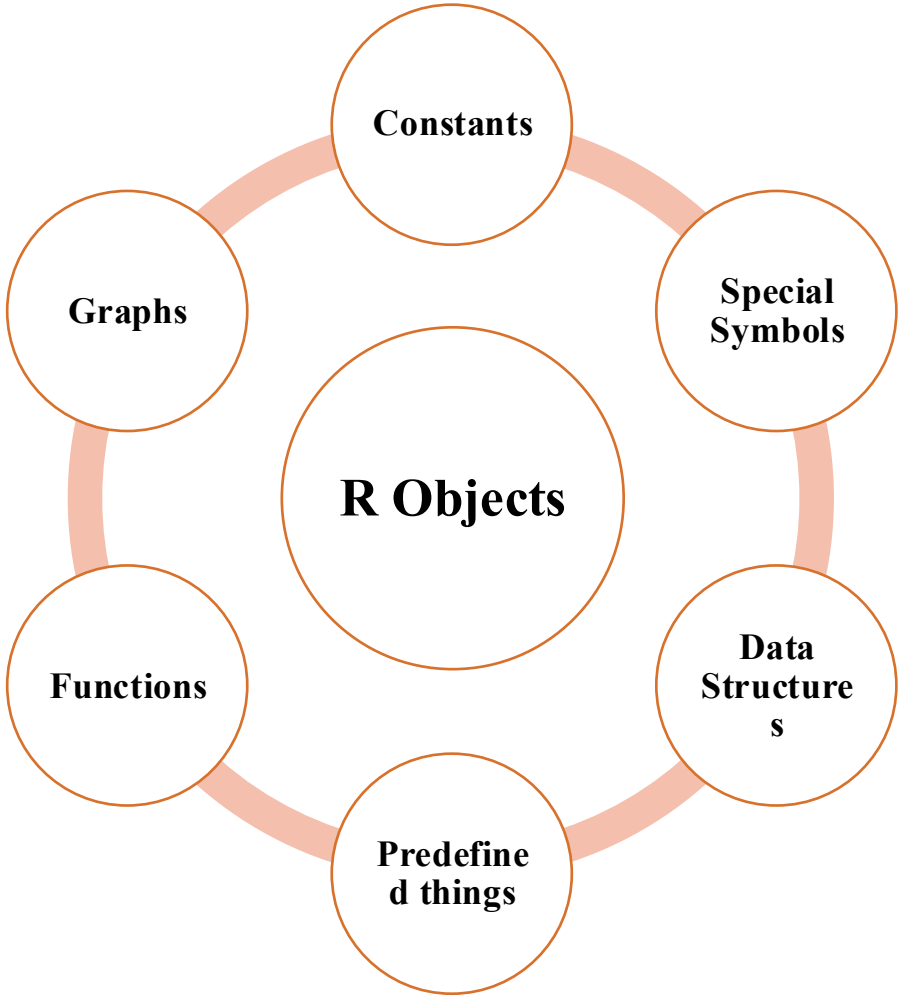


R Objects

An **object** is anything that can be **assigned to a variable**

R objects come in a variety of types. Given an object, the functions **mode** and **class** tell us about its type.

- **mode(object)** has to do with how the object is stored. R has 5 basic modes:
  1. **Numeric**
  2. **Integer**
  3. **Character**
  4. **Logical**
  5. **Complex**
- **class(object)** gives the class of an object. The main purpose of the class is so that generic functions (like **print** and **plot**) know what to do with it.



Data Structures

Vectors

The vector type is really the **heart of R.** Vectors are **one-dimensional arrays** that can hold numeric data, character data, or logical data
The concatenate function **c()** is used to form the vector

```
x<-c(1,2,3)                    #Numeric
y<-c("one", "two", "three")    #Character
z<-c(TRUE, TRUE, FALSE)        #Logical
```

Note that the data in a vector must only be **one type or mode** (numeric, character, or logical)
You can refer to elements of a vector using a numeric vector of positions within brackets
x[c(1,3)] refers to the 1st & 3rd elements of vector x

Matrices

A matrix is a **two-dimensional array** where each element has the **same mode** (numeric, character, or logical). Matrices are created with the matrix function . The general format is

mymatrix <- matrix(***vector***, nrow=***number_of_rows***, ncol= ***number_of_columns***, byrow= ***logical_value***, **dimnames=list(** ***char_vector_rownames, char_vector_colnames***))

- ***y1<-matrix(1:30,nrow=5,ncol=6)***

- ***y2<-1:30***
  dim(y2)<-c(5,6)

- ***y3<-cbind(1:5,6:10,11:15,16:20,21:25,26:30)***

You can identify rows, columns, or elements of a matrix by using subscripts and brackets

- ✓ ***y1[2,]*** ***#Second row of matrix***
- ✓ ***y1[,3]*** ***#Third column of matrix***
- ✓ ***y1[2,3]*** ***#Element in the second row & third column of matrix***
- ✓ ***y1[3,c(1,5)]*** ***#Elements in third row & 1st & 5th columns of matrix***

Arrays

Arrays are similar to matrices but can have **more than two dimensions**. They're created with an array function of the following form:

myarray <- array(***vector, dimensions, dimnames***)

```
dim1<- c("A1", "A2")
dim2<- c("B1", "B2", "B3")
dim3<- c("C1", "C2", "C3", "C4", "C5")
z<- array(1:30, c(2,3,5), dimnames=list(dim1,dim2,dim3))
```

Like matrices and vectors, they must be of a **single mode.**

Extracting elements of an array is similar to that of matrices

Data Frames

The **most common** data structure in R is data frame. A data frame is **more general** than a matrix in that **different columns** can contain **different modes of data** (numeric, character, etc.)

A data frame is created with the **data.frame()** function :

mydata.frame <- data.frame***(col1, col2, col3,…)***

where ***col1, col2, col3***, … are column vectors of **any type** (such as character, numeric, or logical)

```
index<- c(1,2,3)
name<- c("Peter", " Roger", "Hansen")
marks<- c(83, 56, 95)
marksheet<- data.frame( index, name, marks)

marksheet[ ,1:2]                    # Extracting first 2 columns
marksheet[1:2, ]                    # Extracting first 2 rows
marksheet[ , c("name", "marks")]    # Extracting columns using column names
marksheet$marks                     # Extracting a particular column of a data frame
```

Lists

Lists are the **most complex** of the R data types. Basically, a list is an **ordered collection of objects** (components). A list may contain a **combination** of vectors, matrices, data frames, arrays and even other lists.

You create a list using the **list()** function :

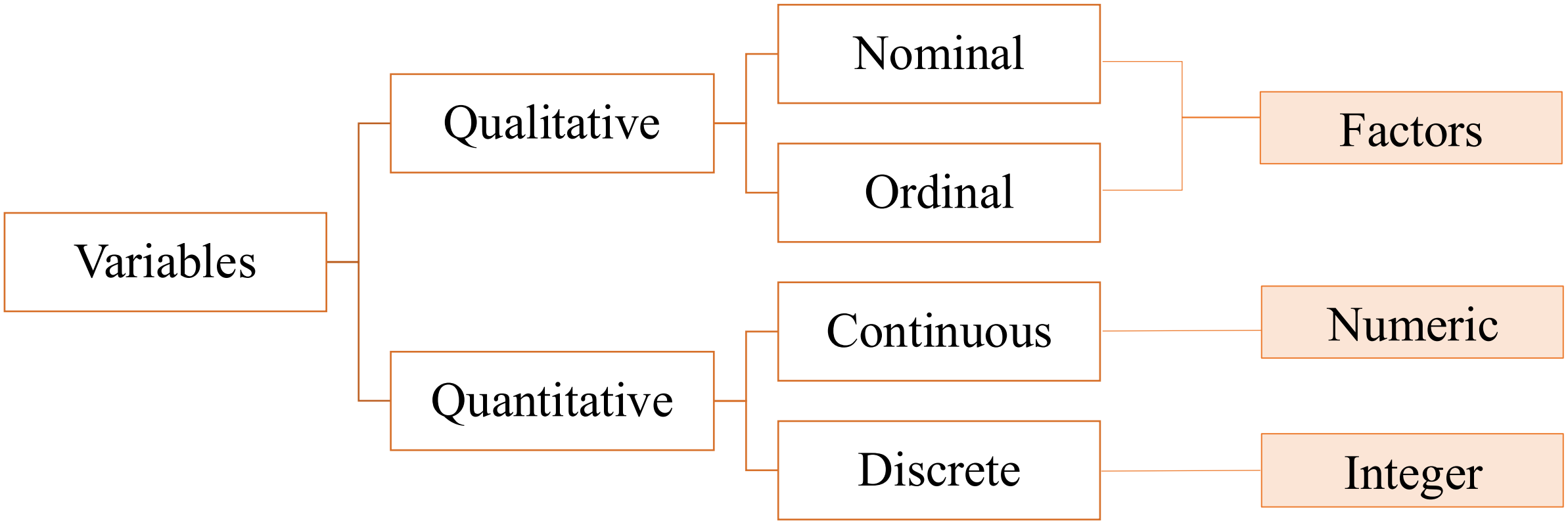
mylist <- list***(object1, object2, …)*** **or** ***mylist*** <- list***(name1=object1, name2=object2, …)***

listR<-list(title="First list", x= c(1,4,5), matrix(1:10,2,5), marksheet=marksheet)

Note, extraction of elements from list is slightly different.

```
listR[[2]]          # Extracting second object of list
listR[["x"]]        # Extracting object by name
listR$x             # Alternate way of extracting single object of list
listR[[2]] [3]      # Extracting a particular element in an object of the list
listR[c(1,3)]       # Extracting multiple objects
```

Factors


Variables
Qualitative
Quantitative
Nominal
Ordinal
Continuous
Discrete
Factors
Numeric
Integer

Categorical (nominal) and **ordered categorical (ordinal)** variables in R are called **factors**

The function **factor()** stores the categorical values as a vector of integers internally. The **factor** function is used to create a factor.

.

Nominal data:

```
data <-  c("Type1", "Type2", "Type1", "Type1“,"Type3")
fdata <- factor(data)
rdata <- factor(data,labels=c("I","II","III"))
```

Ordinal data:

```
status <- c(“Ailing", "Improved", "Excellent", “Ailing")
status <- factor(status, order= TRUE, levels=c("Poor", "Improved", "Excellent"))
```

assigns the levels of the factor variable as 1=Poor, 2=Improved, 3=Excellent.

Data Input

Keyboard

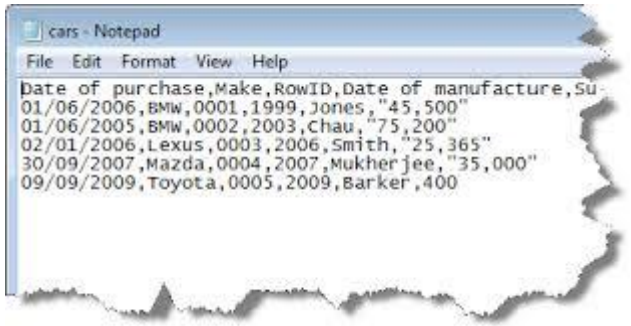
Simplest method of data entry. The **edit()** function in R will invoke a text editor that will allow you to enter your data manually. Here are the steps involved:
1 Create an **empty data frame** (or matrix) with the variable names and modes you want to have in the final dataset.
2 Invoke the text editor on this data object, enter your data, and save the results back to the data object.

keyboard.data<- data.frame(name=character(0),gender=character(0),age=numeric(0),marks=numeric(0))
keyboard.data<- edit(keyboard.data) #Alternative for this line: fix(keyboard.data)

If you click on a column title, the editor gives you the option of changing the variable name and type (numeric, character). You can add additional variables by clicking on the titles of unused columns.

Note: This method of data entry works well for **small datasets**

Delimited Text File



Use **read.table** function

mydataframe <- read.table (file, header=logical_value, sep="delimiter", row.names="name")

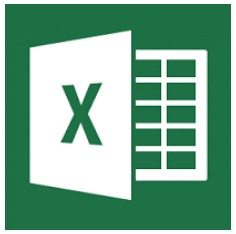
where file is a **delimited ASCII file** , header is a logical value indicating whether the first row contains **variable names** (TRUE or FALSE), sep **specifies the delimiter** separating data values, and row.names is an optional parameter specifying one or more variables to represent row identifiers

cars.data <- read.table("C:\\Users\\Suprit\\Desktop\\cars.csv", header=TRUE, sep=",", row.names="ID")

Note:

- Use **"\\" or "/"** for specifying file path in R
- The default separator in read.table is **sep="",** which denotes **one or more spaces**, tabs, new lines, or carriage returns
- By default, character variables are converted to factors. Including the option **stringsAsFactors=FALSE** will turn this behaviour off for all character variables
- You can use the **colClasses** option to specify a class (for example, logical, numeric, character, factor) for each column.

EXCEL Spreadsheets



AVOID IMPORTING DATA IN EXCEL FORMAT !!!

Reasons:

- Cells could be merged, colored, bold etc.
- Several data tables could be put anywhere
- Charts inside sheets

For more issues with importing excel files go to :http://biostat.mc.vanderbilt.edu/wiki/Main/ExcelProblems

Solution:

Save your excel data in a pure text format : **CSV or tab-delimited**

CSV:

mycsv <- read.csv(file, header=logical_value) or *mycsv <- read.table (file, header=logical_value, sep=",")*

Tab-delimited:

mytab<- read.delim(file, header=logical_value) or *mytab <- read.table (file, header=logical_value, sep="\t")*

SPSS

SPSS datasets can be imported into R via the **read.spss()** function in the **foreign** package . Alternatively, you can use the **spss.get()** function in the **Hmisc** package .

```
install.packages("Hmisc")
library(Hmisc)
myspssdata <- spss.get("mydata.sav", use.value.labels=TRUE)
```

SAS

A number of functions in R are designed to import SAS datasets, including **read.ssd()** in the **foreign** package , **sas.get()** in the **Hmisc** package .and **read.sas7bdat** in **sas7bdat** package. But it is recommended to save SAS datasets in **csv** format

SAS program:

```
proc export data= mydata
outfile ="mydata.csv"
dbms =csv;
run;
```

R program:

```
mydata <- read.table("mydata.csv", header=TRUE, sep=",")
```

Annotating datasets

Variable labels

Use **names** function to edit variable names in data frame

x<- data.frame (a=c(21,33,43),b=c("Jeffrey", "Pat", "Maurice"), c=c("M", "M", "M"))
names(x) <-c("Age", "Name", "Gender")

Value labels

The **factor()** function can be used to create value labels for **categorical variables**

- *Nominal data*

patientdata$gender <- factor(patientdata$gender, levels = c(1,2),labels = c("male", "female"))

- *Ordinal data*

student$grade <- ordered(student$grade, levels = c(1,3, 5), labels = c("Low", "Medium", "High"))

Vectors

Fundamental data type in R

- ***Adding/Deleting***

  x<- c(23,12,43,1)

  x<- c(x[1:3],456,x[4]) #Insert 456 before 1

Always reassign the vector while editing/appending the vector

- ***Obtaining length of vector***

  length(x)

Length of a null vector is 0

- ***Declaration***

  y<- c(5,12,13)

  #Alternative

  y<- vector(length=3)

  y[1]<-5

  y[2]<-12

  y[3]<-13

Do not need to mention the variable type of the vector like C

- ***Vector Arithmetic***

```
x<-c(1,2,3)
y<-c(2,4,6)
x+y        #addition
x*y        #multiplication
y/x        #division
x^y        #exponentiation
y%%x       #just the remainder
y%/%x      #Integer part of the quotient
```

R can be used as a calculator. Note that the operations occurs element by element

- ***Vector Indexing***

```
x<-c(1,4,5,12,-9)
x[c(1,3)]        #Extract 1st & 3rd elements of x
x[1:3]           #Extract 1st , 2nd & 3rd elements of x
x[-5]            #Exclude the 5th element of x
x[-c(2,4)]       #Exclude 2nd & 4th element of x
```

Frequently used operation. Negative subscripts mean that we want to exclude elements in our output

- ***Generating vectors***

```
x<- c(2,4,6,8,10,11,11,11,11,11.5,11.5,11.75,11.75,12,13,14,15)
x1<- 12:15                          # ":" Produces consecutive integers
x2<- seq (from=2,to=10,by=2)        # "seq" generates a AP sequence
x2.1<-seq(from=2,to=10,length=5)
x3<- rep(11,times=4)                # Repeat constants
x4<- rep(c(11.5,11.75),each=2)

x<- c(x2, x3,x4,x1)
```

Very useful in data creation. Frequently required

- ***Using all() and any()***

```
x<-1:15
any(x>4)
all(x>4)
```

any() & all() functions are shortcuts to report whether any or all of their arguments are TRUE

- ***Vectorized Operations***

One of the most effective ways to achieve speed in R code is to use operations that are **vectorized.** Meaning that a function applied to a vector is actually **applied individually to each element**.

```
u<- c(5,2,8)
v<- c(1,3,9)
u>v                #Checking condition element wise
sum(u>v)           #Number of instances where condition is  TRUE
sqrt (1:9)         # Mathematical functions are vectorized
w<- c(1.2,3.9,0.4)
round(w)
u+4
```

In R, vectorization is often done for both (a) readability and (b) performance

- ***NA & NULL***

```
x<-c(12,NA,20,28)
mean(x)
mean(x,na.rm=TRUE)
y<-c(12,NULL,20,28)
mean(y)
```

Missing data in R is represented by NA. NULL on the other hand represents non existent values

- ***Filtering***

This allows us to **extract vector elements** that satisfy certain conditions. Very useful in data manipulation.

```
x<-c(2,6,3,8,10)
y<- x>5
x[y]
# Alternative
x[x>5]
```

Extraction of vector elements based on multiple conditions is also possible

```
price<-c(10,30,20,50)
sales<-c(1000,200,500,100)
price[(sales*2)>300]
```

Here we are using one vector to determine indices to use in filtering another vector

```
z<-c(1,3,8,2,20)
z[z>3]<-0
z
```

Assignment can be compactly done based on certain conditions easily

- ***Filtering with the subset() function***

```
x<-c(2,6,3,8,10)
subset(x, x>5)
y<- c(6,1:3,NA,12)
y[y>5]
subset(y, y>5)
```

Difference between ordinary filtering & this is the manner in which NA values are handled

- ***Filtering using which() function***

```
x<-c(2,6,3,8,10)
which(x>5)
x[which(x>5)]
```

which() allows us to find out the positions that satisfy the given conditions

- ***Vectorized ifelse() function***

In addition to the usual if-then-else construct found in most languages, R includes a vectorized version, the **ifelse()** function

ifelse(b, u, v)

where b is a **Boolean** vector , u & v are vectors

x<-2:10
ifelse(x%%2==0, "Even", "Odd")
ifelse(x>5, x+2, x+3)

Advantage over standard if-else construct is that it is vectorized, thus potentially much faster

- ***Vector Element Names***

x<-c(1,2,4)
names(x)
names(x)<- c("a", "b", "c")
names(x)
names(x)<-NULL

Assign vector element names using names() function. Remove names by assigning NULL

Matrices

A **matrix** is a **vector** with two additional attributes: number of rows & number of columns. Matrices are special cases of arrays which can be **multidimensional.**

- ***Creating Matrices***

  Matrix row & column subscripts begin with 1. Ex: a[1,1] is the first element of the matrix a

  Use **matrix()** function:

```
y<- matrix(c(1,2,3,4), nrow=2, ncol=2)
#Alternative
y<- matrix(nrow=2,ncol=2)
y[1,1]<-1
y[2,1]<-2
y[1,2]<-3
y[2,2]<-4
y1<-matrix(c(1,2,3,4),nrow=2,ncol=2,byrow=TRUE)
#Alternative
y2<- cbind(c(1,2),c(3,4))
y3<- rbind(c(1,3),c(2,4))
```

When we construct a matrix directly with data elements, the matrix content is filled along the column orientation by default

The byrow argument enables input to be read along row orientation form

cbind() & rbind() combines vectors, matrices, data frames by column & row respectively

- ***Matrix Computations***

Most matrix operations use the same symbols as the math operations

```
mat1<-matrix(1:6,nrow=2,ncol=3)
mat2<-matrix(1:6,nrow=2,ncol=3,byrow=TRUE)
t(mat1)                    # Transpose
mat1+mat2                  # Addition
mat1-mat2                  # Subtraction
mat3<-mat1 %*% t(mat2)     # Matrix multiplication
det(mat3)                  # Determinant
solve(mat3)                # Inverse
mat1*2                     # Element wise multiplication
mat1*mat2
mat1/2                     # Element wise division
mat1/mat2
```

Note the difference in symbols used for matrix multiplication & element wise multiplication

- ***Matrix Indexing***

```
mat<- matrix(1:12,3,4)
mat[2:3,]                  # Extract 2nd to 3rd row
mat[, c(1,3)]              # Extract 1st & 3rd column
mat[,-4]                   # Exclude 4th column
```

Similar to vector indexing

- ***Filtering***

```
mat<-matrix(c(1,2,2,3,3,4),nrow=3,ncol=2,byrow=TRUE)
i<-mat[ ,2]>=3
mat[i, ]
# Alternative
mat[mat[,2]>=3, ]

z<-c(5,12,13)
x[z%%2==1, ]

m<-matrix(1:6,2,2)
m[m[ ,1]>1 & m[ ,2]>5, ]
m[m[ ,1]>1 & m[ ,2]>5, ,drop=FALSE]

m<- matrix(c(5,2,9,-1,10,11),3,2)
which(m>2)
```

Matrix filtering is similar to vector filtering

Filtering criterion can be based on a different variable from the one to which filtering will be applied

The drop=FALSE argument retains the two dimensional nature of the data

- ***Using the apply() function***

One of the most **famous** & **popular** features is the **apply()** family of functions, such as **apply(), lapply(), sapply(), mapply(), tapply().** Let us look at apply() which is used to evaluate a function over **the margins** of a matrix.

apply(m, dimcode, f, fargs) where

- m is the matrix
- dimcode is the dimension, 1 if function applies to rows, 2 for columns
- f is the function to be applied
- fargs is an optional set of arguments to be supplied to f

```
m<-matrix(1:6,3,2)
apply(m,2,mean)                    #Column means
apply(m,1,max)                     #Row maximum
x<-matrix( rnorm(200),20,10)
apply(x,1,quantile,probs=c(0.25,0.75))#1st & 3rd quartile of each row
```

Do not loop , use apply() in R

Makes code compact, easier to read, modify and avoid possible bugs

- ***Adding/Deleting matrix rows & columns***

```
mat<- matrix(1:12,3,4)
one<- c(1,1,1)
mat<-cbind( one, mat)
odd<- c(3,5,7,9,11)
mat<-rbind( mat, odd)
```

cbind() & rbind() functions let you add columns & rows to a matrix

```
mat<-mat[c(1,3), ]
```

Delete columns/rows by reassignment

- ***Naming matrices/Other functions***

```
mat<-matrix(1:4,2,2)
colnames(mat)
colnames(mat)<-c("a", "b")
rownames(mat)<-c("Row1", "Row2")
mat
dim(mat)          #Get dimensions of matrix
nrow(mat)         #Alt: dim(mat)[1]
ncol(mat)         #Alt: dim(mat)[2]
```

Similar to names() function for vectors

Useful while writing functions whose argument is a matrix

Lists

A list is a **collection of objects**. The elements of a list can be **different types** of object and can be of **different lengths**. Plays a central role in R and forms a basis for data frames & object oriented programming.

- ***Creating Lists***

```
x<-list( first=1:10, second=c("yes", "no"), third=c(T,F), fourth= gl(2,3) )
x1<-list(1:10, c("yes", "no"), c(T,F), gl(2,3) )
#Alternative
y<-vector(mode="list")
y[["first"]]<-1:10
y[["second"]]<-c("yes","no")
y[["third"]]<-c(T,F)
y[["fourth"]]<-gl(2,3)
```

Using names in lists makes it clearer & less error prone

Lists are also vectors so they can be created via vector()

- ***Indexing***

```
x$first              #Extract component of list
x[[1]]               # Alternative
x[["first"]]         # Alternative
x[1:3]               #Extract more than 1 component
x[["first"]][5]      #Extract a particular element of component
```

Use [[ ]] to access list components

- ***Adding/Deleting List Elements***

```
z <-list(a="abc", b=12)
z$c <-"New"     #Adding
z[[4]] <-27
z[5:7] <-c(FALSE,TRUE,FALSE)

z$b <-NULL     #Deleting

z1<-c( z, list(last=matrix(1:6,3,2))) #Concatenate lists
```

- ***unlist() & names() function***

```
l<-list(a=1:100,b=2:5)
names(l)                    #Get tags of list
unlist(l)                   # List to vector
list1<-list(name="Joe", salary=5500,Graduate=TRUE)
unlist(list1)
names(list1)<-NULL   #Remove names from list
#Alternative
list1<-unname(list1)
```

Adding/Deleting list elements are often required in R

Indices of list get adjusted after deletion

Concatenating lists is another way for adding list elements

unlist() function flattens the list into a vector, and coerce types so that all elements are of the same type

- ***lapply() & sapply() functions***

Two functions are handy for applying functions to lists: **lapply** and **sapply**

```
x<-list(a=1:5,b=rnorm(100), c=c(23,45,67))
lapply(x, median)
lapply(x, function(y) (max(y)+min(y))/3)
lapply(x, function(y) c(min(y),max(y),mean(y),median(y)))
```

lapply() loops over a list and evaluates a function on each component

```
sapply(x, median)
sapply(x, function(y) (max(y)+min(y))/3)
desc<-sapply(x, function(y) c(min(y),max(y),mean(y),median(y)))
rownames(desc)<-c("Min", "Max", "Mean", "Median")
```

sapply() is a user-friendly version of lapply() by default returning a vector or matrix if appropriate

lapply() is the **workhorse** of many other *apply functions. Peel back their code and you will find lapply underneath.

Data Frames

A data frame is like a matrix, with a two dimensional row & column structure. However each column may have **different modes**. Technically, a data frame is a **list** of equal length vectors. Most of the **real life datasets are data frames**.

- ***Creating data frames***

```
kids=c("Jack", "Jill")
ages<-c(12,10)
d<-data.frame( kids, ages, stringsAsFactors=FALSE)
```

By default all character vectors are converted to factors while creating data frames

- ***Accessing data frames***

```
#Since data frame is a list
d[[1]]                          #First column
d$kids
#Since data frame is matrix like
d[ , 1]                         # First column
d[ , colnames(d)=="kids"]
d[ , "kids"]
```

It is clearer & safer to extract columns by column names

- ***Filtering***

```
data<-read.table("C:\\Users\\Suprit\\Desktop\\example.txt",header=TRUE)
data[2:5, ]                    # 2nd to 5th rows
data[2:5,4]                    # 4th column of above
data[2:5,4,drop=FALSE]         # Retains the data frame structure
data[ data$maths >80 & data$statistics >90, ]
#Alternative
subset(data, maths >80 & statistics >90)
```

Mostly imported data sets are stored as data frames

- ***rbind() & cbind()***

```
kids=c("Jack", "Jill")
ages<-c(12,10)
d<-data.frame( kids, ages, stringsAsFactors=FALSE)
d<-rbind(d, list(" Laura ",19))
d<-cbind(d, c(89,45,76))
names(d)[3]<-"Marks"
```

rbind() & cbind() are used to append data frames

- ***apply, lapply & sapply***

```
data<-read.table("C:\\Users\\Suprit\\Desktop\\example.txt",header=TRUE)
apply(data[ ,3:5],1,max)
data1<-lapply(data, sort)
data2<-sapply(data[ ,3:5], function(x) c(sum(x),mean(x)))
rownames(data2)<-c("Total Marks", "Average Marks")
```

Since data frame is both a list & matrix like we are able to apply all three functions

- ***attach(), detach() & with()***

The **attach()** function **adds** the data frame to the R search path. The **detach()** function **removes** the data frame from the search path.

```
attach(data)
statistics
maths + computer
detach(data)
#Alternative
with(data, {
        print(statistics)
        print(maths + computer)})
```

The attach() function in R makes objects within data frames accessible in R with fewer keystrokes. Should be avoided since there may be multiple objects with same name.